

0.28. Aller à une case : va_a

`va_a(x, y, tuile, colonne, ligne)` : on ne compte plus les pas, on dit où aller.

1. Ce qu'il faut comprendre

Jusqu'ici, on comptait les pas. Pour aller en (10, 5) depuis (3, 2), il faudrait calculer $10 - 3 = 7$ et $5 - 2 = 3$.

`va_a` fait ce calcul pour nous.

Ce qui est nouveau ici : `va_a(colonne, ligne, tuile, versColonne, versLigne)`. Les deux derniers nombres ne sont plus des pas, mais **la case d'arrivée**.

Le chemin : à chaque quart de seconde, un pas vers la colonne voulue (si elle n'y est pas) **et** un pas vers la ligne voulue (si elle n'y est pas). De (0, 0) à (10, 5) : en **diagonale** jusqu'en (5, 5), où la ligne est bonne, puis **tout droit** jusqu'en (10, 5).

Les positions sont rangées toutes seules, comme avec `deplace` : écris-la seule sur sa ligne.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <va_a>` → [0.27.3. La fonction va_a\(\) — aller jusqu'à une case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1149 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	62 o	← la console, à l'allumage — la mise en route de la console
<code>va_a()</code>	176 o	← <code>main()</code> — une fonction fournie par le langage
<code>main()</code>	56 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← <code>EffacerCases</code> , <code>EcrireNombre</code> — attend de pouvoir écrire une case
AdresseCase	10 o	← <code>poser()</code> , <code>effacer()</code> — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← <code>effacer()</code> — efface des cases
EcrireNombre	33 o	← <code>nombre()</code> — écrit un nombre en chiffres
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← <code>CopierTuiles</code> — la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>poser()</code>	× 2	26 o sur place, dans <code>va_a()</code> + <code>AdresseCase</code> (10 o)
<code>nombre()</code>	× 2	20 o sur place, dans <code>main()</code> + <code>EcrireNombre</code> (33 o)
<code>effacer()</code>	× 1	15 o sur place, dans <code>va_a()</code> + <code>AdresseCase</code> (10 o), <code>EffacerCases</code> (12 o)
<code>image()</code>	× 2	6 o sur place, dans <code>va_a()</code> , <code>main()</code> + <code>AttendreImage</code> (28 o)

Tes fonctions appelées :
va_a() x 1 23 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <va_a> // mène une tuile jusqu'à une case
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
#include <nombre> // écrit un nombre en chiffres
```

2. UNE VARIABLE — X

```
uint8_t x = 0; // la COLONNE de la lettre
```

3. UNE VARIABLE — Y

```
uint8_t y = 0; // la LIGNE de la lettre
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // La ligne, morceau par morceau :
    //
    // va_a(x, y, ALPHABET[0], 10, 5);
    //      | | |           | |
    //      | | |           | --- la LIGNE d'arrivée : 5
    //      | | |           +----- la COLONNE d'arrivée : 10
    //      | | +----- tuile : la lettre A
    //      +--+----- d'où elle part ; la console y range
    //                  l'arrivée : x = 10, y = 5
    //
    // A                (0, 0)
    //  \
    //   \              la diagonale, tant que la ligne n'est pas bonne...
    //    \
    //     \
    //      \
    //       → → → → A ...puis tout droit : (5, 5) → (10, 5)
    va_a(x, y, ALPHABET[0], 10, 5);
    nombre(0, 17, x); // 010
    nombre(4, 17, y); // 005

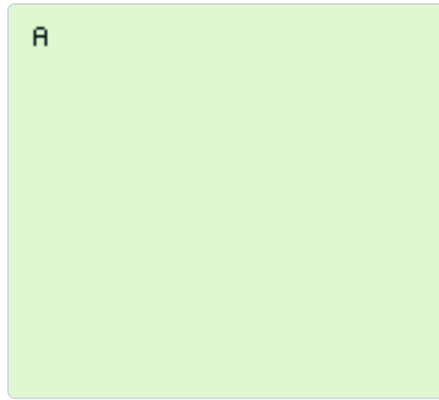
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

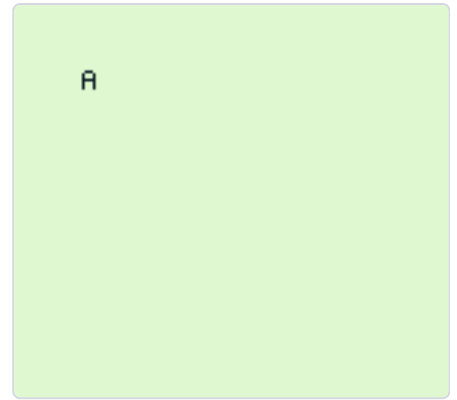
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



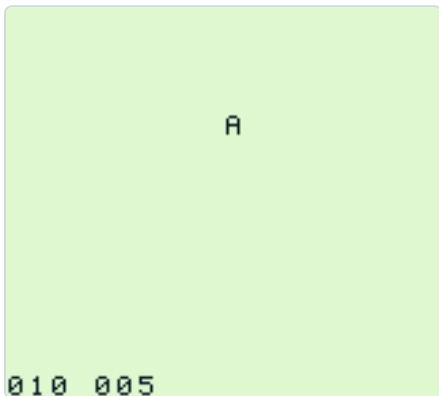
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui descend en biais jusqu'en (5, 5), puis file tout droit jusqu'en (10, 5).

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la diagonale d'abord : (3, 3), (5, 5)
- ✓ puis tout droit : (8, 5)
- ✓ arrivé en (10, 5) ; $x = 10$, $y = 5$ (10,5)

Ensuite : **0.28.1. Aller à une case : `va_a` — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++